

JavaScript

du ECMAScript au DOM

Sébastien Bougleux
Normandie Univ, UNICAEN
IUT Cherbourg-Manche
Département MMI

CM1

- 1 Introduction**
- 2 JavaScript et (X)HTML**
- 3 Bases d'ECMAScript**
- 4 Objets**
- 5 Arguments des fonctions**

À chaque CM est associé un QCM noté à réaliser sur ECAMPUS

1 - Introduction

JavaScript

- langage de **programmation de scripts**
- langage **interprété**
 - scripts convertis en instructions exécutables par un interprète (programme en exécution)
 - exemples de langages interprétés : BASIC, PHP, Ruby, Perl, JavaScript, ...
 - différent des langages compilés (C, C++, ...)
ou des langages semi-interprétés (Java)
- **dialecte de l'ECMAScript**
pour la création de documents HTML dynamiques

JavaScript

- créé pour la **conception de pages HTML/CSS** et le **contrôle d'objets instanciés sur des clients** (initialement des navigateurs Web)
- permet de programmer des traitement évolués
 - dans des documents Web à travers les navigateurs
 - plus généralement dans toute application disposant d'un interpréteur
- libre et multiplateformes
- intégré aux navigateurs Web depuis *Microsoft Internet Explorer* (1996)
- fondement des applications Web 2.0
 - permet de créer des applications dynamiques en-ligne reliant entre-eux des objets et des ressources sur des clients et/ou des serveurs

Historique non-exhaustif

- initié par Brench Eich (*Netscape Communication Corporation*)
 - **Mocha**, puis **LiveScript** et finalement **JavaScript**
- 1995 : *Sun Microsystems* et *Netscape* annoncent la sortie de **JavaScript**
- 1996 : JavaScript sert à la fondation d'**ECMAScript**
 - standard **ECMA-262** développé par **ECMA**
 - JavaScript devient un dialecte d'ECMAScript
 - ECMA = *European Computer Manufacturers Association*
 - site ECMA : <http://www.ecma-international.org/>
- 1996 : *Microsoft* sort **Jscript**
 - dialecte d'ECMAScript
- 1997 : 1ère version d'**ECMA-262**
- 1998 : **ECMAScript** adopté comme **standard** ISO/IEC-16262
 - *International Organization for Standardization* (ISO)
 - *International Electrotechnical Commission* (IEC)

ECMAScript

- fondé sur la spécification **ECMA-262**
- **langage interprété**
 - basé objets
 - à prototypes
- **spécification** décrit
 - syntaxe et mot-clés
 - convention et notations
 - types et conversion de types
 - contexte d'exécution
 - expressions et structures de contrôle
 - fonctionnement des fonctions
 - objets natifs
 - concept de prototypes

ECMAScript - langage basé objets

- **objet**

- ensemble de paires (propriété,valeur)
- **propriété** : représentée par un mot (nom de la propriété)
- **valeur** : un autre objet
- les propriétés peuvent être des fonctions

- **exemple**

- un objet Point en 2D a deux propriétés : x et y
- les valeurs associées à x et y sont des nombres, initialisés à 0
- en ECMAScript

```
var Point = {x:0, y:0};
```

- langage **basé-objets**, à **prototype**

- des objets fournissent les bases du langage
- **tout est objet (ou presque)**
- les objets ne sont pas des instances de classes
- les objets sont équipés de constructeurs permettant de générer leurs propriétés
- le **prototypage** permet de générer des objets héritiers personnalisés

ECMAScript et ses dialectes

- **plusieurs dialectes** respectant la spécification ECMA-262
 - **JavaScript** (*Mozilla Firefox, Google Chrome, Konqueror, Safari, ...*)
 - **JScript** (*Microsoft Internet Explorer*)
 - **JScript .NET** (*Microsoft .NET*)
 - **ActionScript** (*Adobe Flash, Adobe Flex*)
 - **VRMLScript**
 - **QtScript** (*KDE*) ...
- enrichissent l'ensemble des objets natifs d'ECMAScript avec d'autres objets provenant de l'environnement d'exécution du script
 - navigateurs Web
 - plus généralement des interpréteurs embarqués dans une application

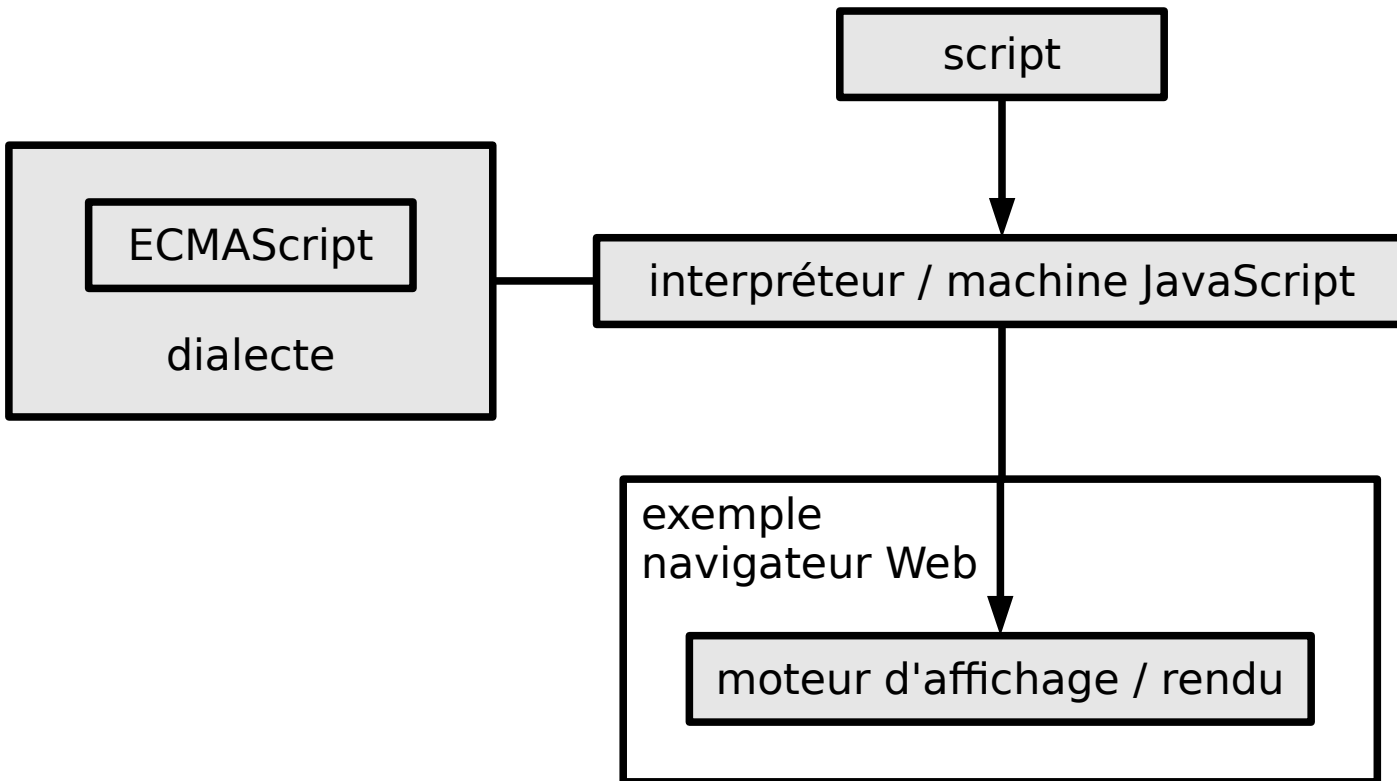
Objectifs du cours

- bases de la spécification ECMA-262, *i.e.* **ECMAScript** (ES)
- bases du **JavaScript** pour le web
 - API DOM et BOM

Remarques

- Javascript implémente ECMAScript
 - plusieurs navigateurs → plusieurs versions/dialectes
- Javascript et JScript (Microsoft) pour le web sont équivalents
 - juste un problème de dénomination (noms déposés)

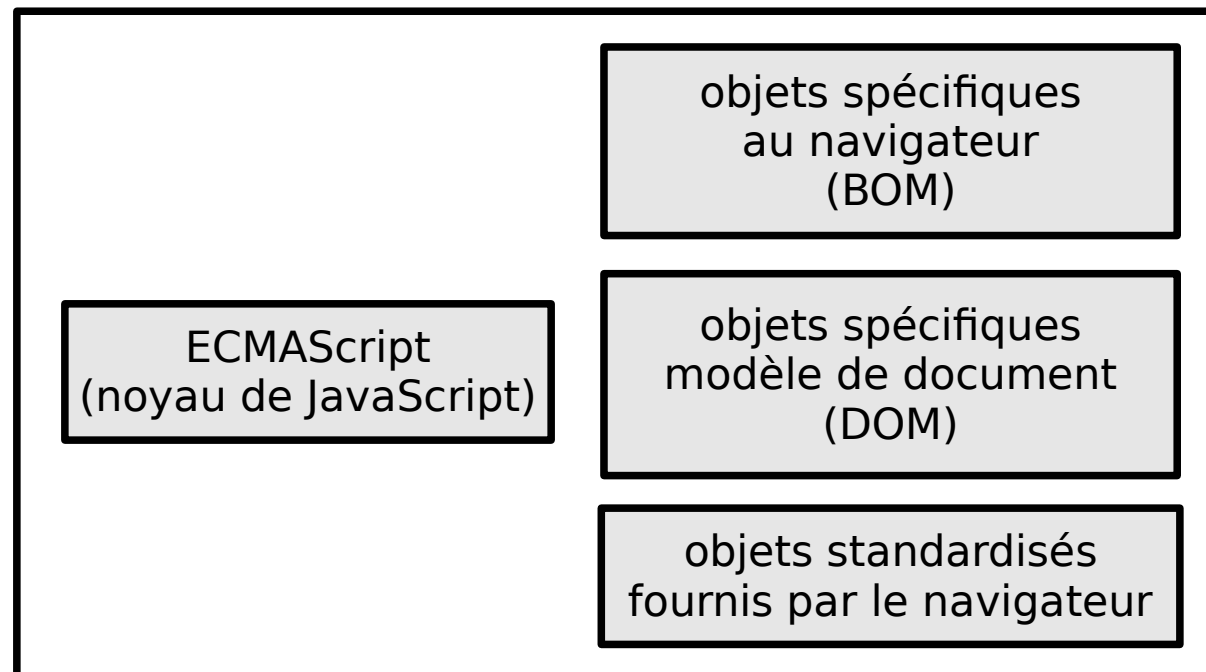
2 - (X)HTML & Javascript



JavaScript et Navigateurs Web

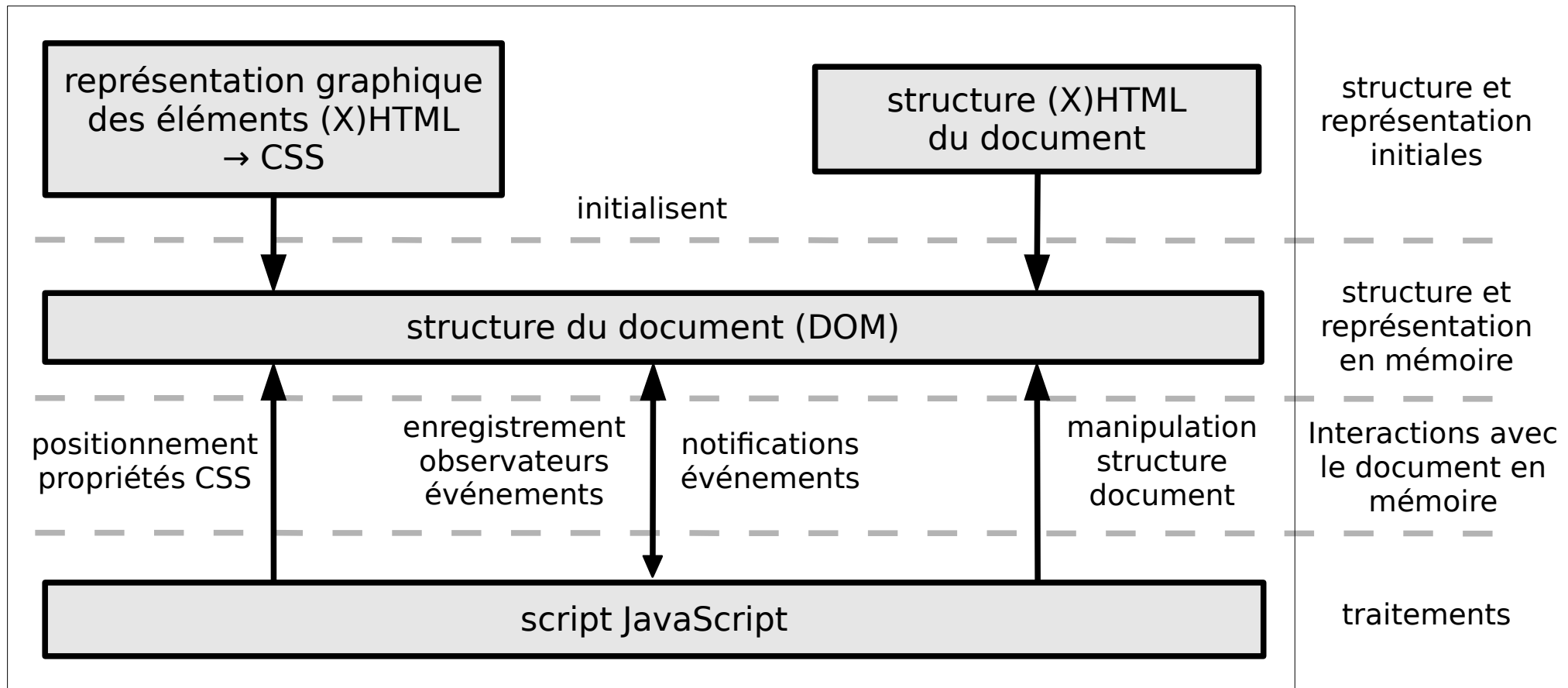
- désigne dans ce cours le JavaScript et le JScript
- bien que non-standardisés
les **objets** du JavaScript sont généralement **normalisés**
 - c'est quasiment le cas des objets relatifs au DOM (*Document Object Model*)
 - ce n'est pas toujours le cas de ceux relatifs au navigateur

éléments JavaScript
présents dans un
navigateur Web



JavaScript et Navigateurs Web

- constituants d'un document Web (X)HTML



JavaScript et Web 2.0

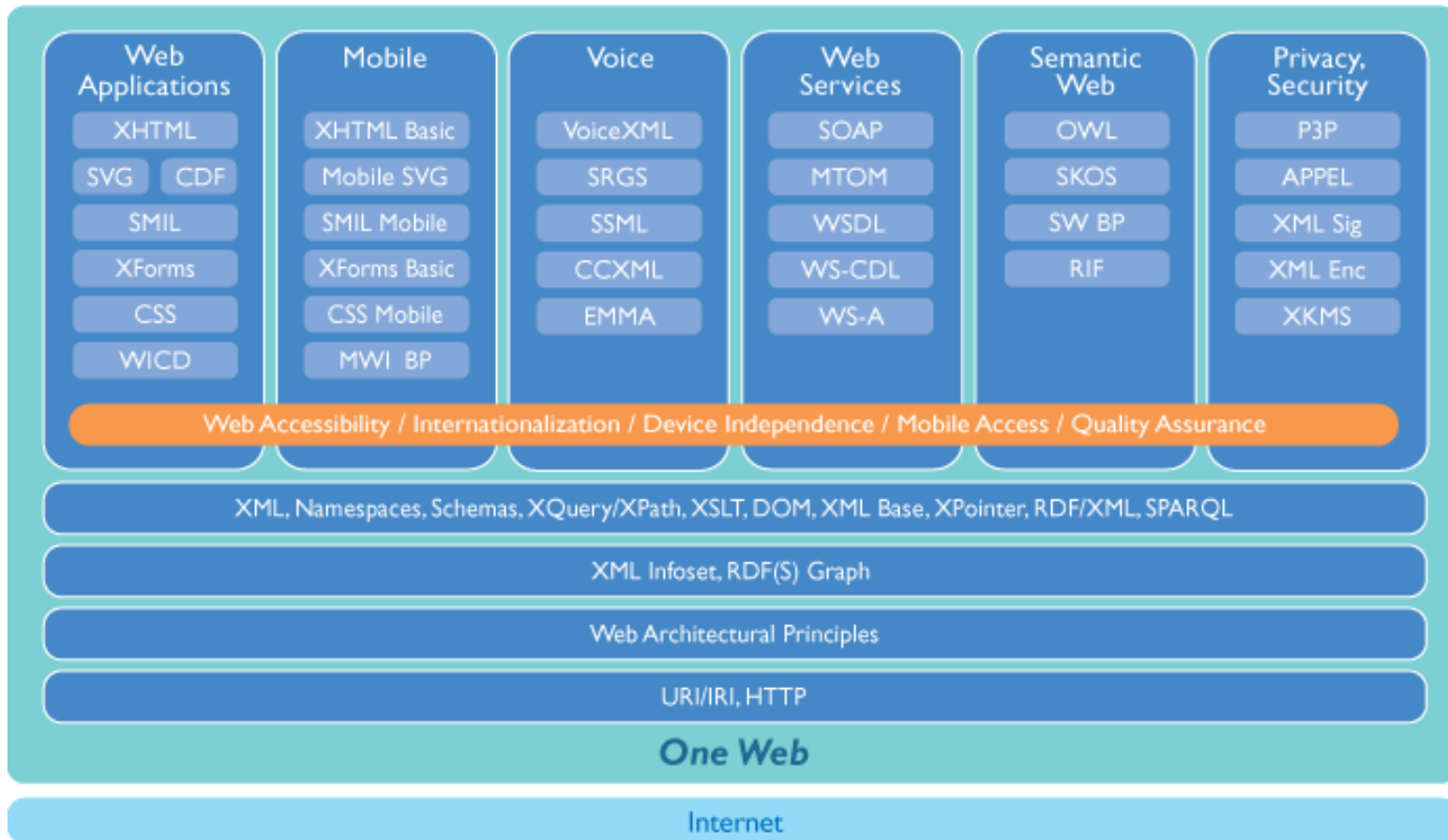
- **enrichissement des interfaces d'applications web**
 - **interactivité**
 - ✓ gestion des événements, déclenchement de traitements
 - ✓ modification de la structure en mémoire
 - ✓ réalisation d'effets graphiques
 - ✓ interaction avec des applications / services accessibles sur un réseau
 - **prise en charge des différents navigateurs**
 - ✓ assurer le même rendu graphique dans les différents navigateurs
 - ✓ intégrer les spécificités du DOM
- **échange de données**
 - réalisation de **requêtes HTTP** (*HyperText Transfer Protocol*)
 - échange de données avec des applications distantes
 - techniques **AJAX** (*Asynchronous JavaScript and XML*)

Lien utile / Référence pour les TD-TP

- JavaScript sur w3schools : <http://www.w3schools.com/js/>

JavaScript et Navigateurs Web

- API Javascript



Déclarer un script en (X)HTML

- identique pour tous les navigateurs
- élément `<script>`

code **interne** au document (X)HTML

```
<script type='application/javascript'>  
    // écriture du code  
</script>
```

ou

```
<script type='application/javascript' src='code.js'>  
</script>
```

code **externe** au document (X)HTML

ou

```
<script type='application/javascript' src='code.js'>  
    // écriture du code en plus  
</script>
```

code **externe**
+ **interne**

Où placer le script dans le code (X)HTML ?

- n'importe où dans l'élément `<head>` et ses descendants
- n'importe où dans l'élément `<body>` et ses descendants
- la place de l'élément `<script>` dans le document joue un rôle important pour l'exécution du script et pour les traitements qu'il réalise
 - ce point sera discuté ultérieurement

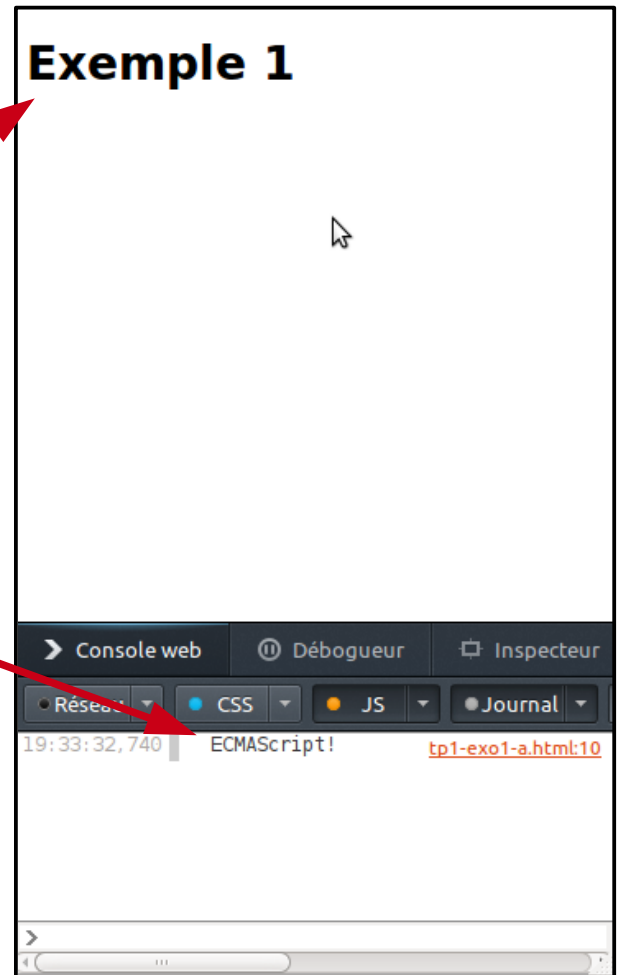
Exemple 1 - script interne

- affichage de données dans la **console web du navigateur**
 - affichage de la chaîne de caractères « ECMAScript ! »

```

<!doctype html>
<html>
  <head>
    <title>Exemple 1</title>
    <meta charset="UTF-8" />
  </head>
  <body>
    <h1>Exemple 1</h1>
    <script>
      console.log("ECMAScript!");
    </script>
  </body>
</html>
  
```

la fonction **log** de l'objet **console** permet d'afficher des chaînes de caractères dans la console du navigateur



Exemple 2 - script externe

- affichage de données dans la **console web du navigateur**
 - affichage de la chaîne de caractères « ECMAScript ! »

```
<!doctype html>
<html>
  <head>
    <title>Exemple 1</title>
    <meta charset="UTF-8" />
  </head>
  <body>
    <h1>Exemple 1</h1>
    <script src='moncode.js'>
    </script>
  </body>
</html>
```

chargement

```
// moncode.js
console.log("ECMAScript!");
```

Type MIME associé à un script

- devrait être **application/javascript**
`<script type='application/javascript'></script>`
 - **valide** avec Firefox, Safari, Chrome
 - **non-valide** avec Internet Explorer version 8 et inférieures
 - **par défaut (aucun type spécifié), le JavaScript est considéré en HTML5**
- en XHTML **application/javascript** est obligatoire
- informations complémentaires : tools.ietf.org/html/rfc4329

ECMAScript

- spécification : <http://www.ecma-international.org/publications/standards/Ecma-262.htm>
- ECMAScript chez ECMA : <http://www.ecma-international.org/memento/TC39.htm>
- ecma-script.org : <http://www.ecma-script.org/>
- **compatibilité ES - navigateurs** : kangax.github.io/compat-table/es5/ **

Javascript

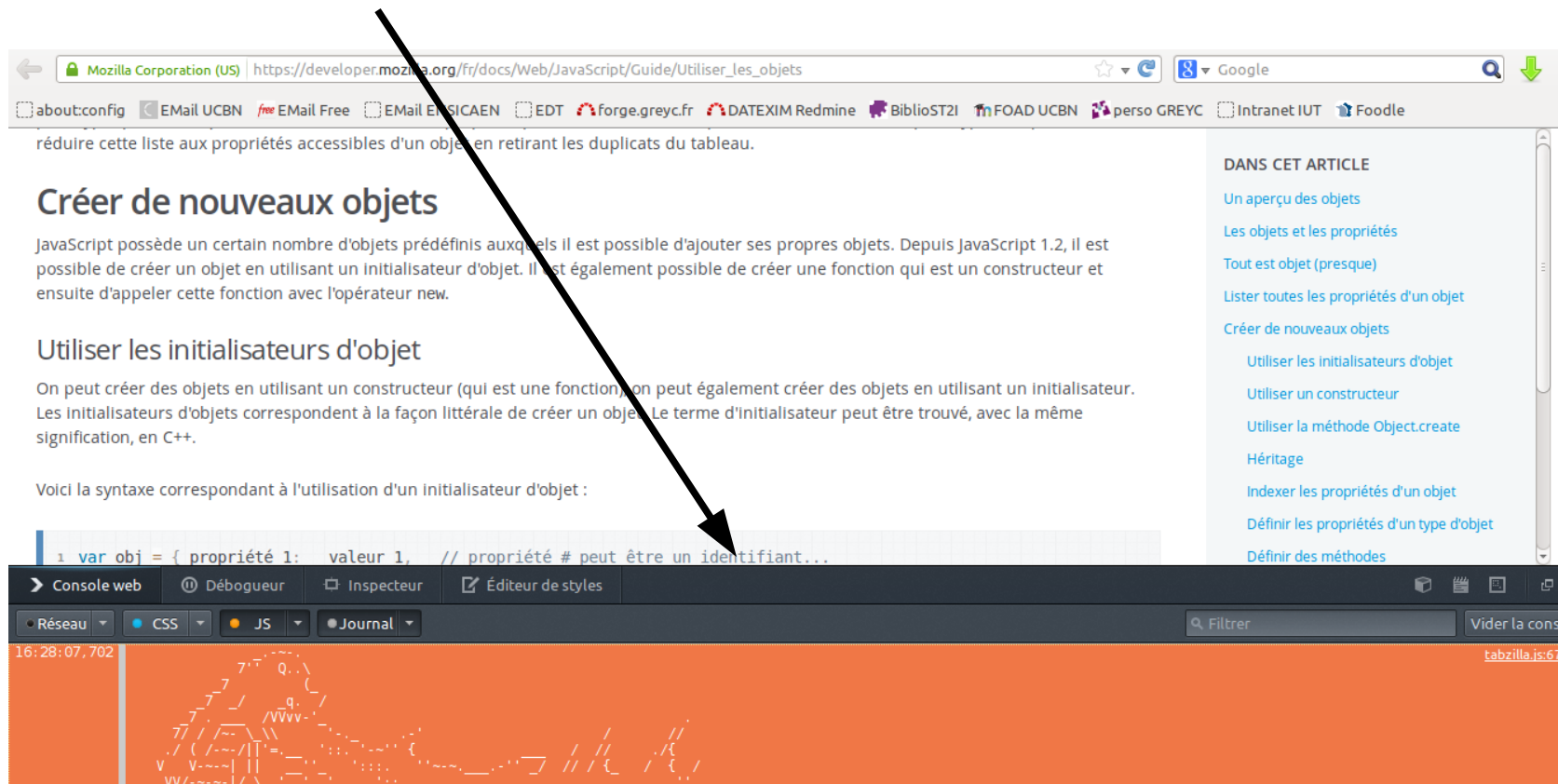
- **Mozilla (MDN)**: developer.mozilla.org/en-US/docs/Web/JavaScript **
- **versions Javascript (1.8.5)**
https://developer.mozilla.org/en-US/docs/Web/JavaScript/New_in_JavaScript

JScript

- **Microsoft (JScript)**
- **référence**
<https://docs.microsoft.com/en-us/scripting/javascript/javascript-language-reference>

Outils utilisés en TP

- écriture du code : éditeur de texte
- tests avec Firefox, Chrome, IE, Safari
 - **et console web du navigateur**



Outils utilisés en TP

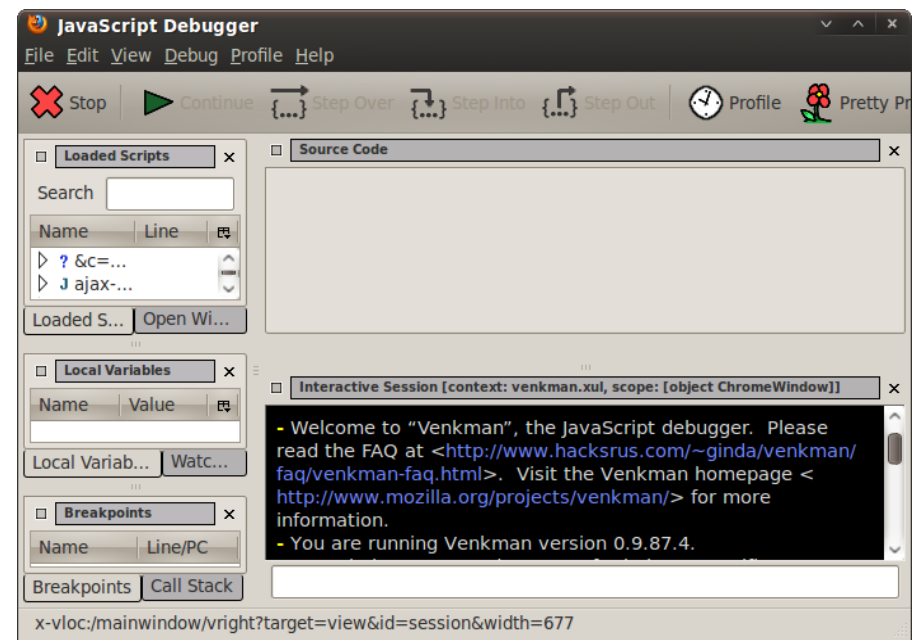
- tests avec Firefox, Chrome, IE, Safari
 - **console d'erreurs**
 - **et gestion des erreurs**

- **Firefox**

- Outils > Console d'erreurs
- *JavaScript Debugger (Venkman)*
- *Firebug* (<http://getfirebug.com/>)

- **Internet Explorer**

- *Microsoft Script Debugger*
- *Microsoft Script Editor*



3 – Bases d'ECMAScript

Variables (1)

- **déclaration** : mot-clé **var** instruction

```
var i;
```

- i est une variable (**sans type explicite**)
- le type est déterminé lors de l'**affectation**

```
var i;    // déclaration  
i = 10;   // affectation
```

commentaires
en ligne

- i représente un nombre
- **réaffectation** possible avec un autre type

```
var i;                // déclaration  
i = 10;               // affectation  
// faire qq chose avec i  
// ...  
i = "ECMAScript!";    // affectation+changement de type
```

- i représente maintenant une chaîne de caractères

- **ECMAScript est faiblement typé**

Variables (2)

- **déclaration** et **affectation** en 1 seule instruction

```
var i = 10;
```

→ i est une variable qui représente un nombre

- **déclaration seule**

```
var i;
```

→ i a été affectée implicitement à la valeur **undefined**

→ équivalent à

```
var i = undefined;
```

- tester si une variable a été affectée

```
if (i !== undefined)
{
    // faire qq chose avec i
}
```

Variables (3)

- **déclaration implicite** et **affectation** sans le mot-clé **var**

```
i = 10;
```

- *i* est une variable qui représente un nombre initialisé à 10
- pas de mot-clé **var**
- **affectation obligatoire**
- **syntaxe DANGEREUSE à utiliser avec précaution**
nous reviendrons sur ce point

Types intégrés

Type intégré / Objet	valeur(s)
Undefined	undefined
Null	null
Boolean	true false
Number	▪ double précision, 64-bits, format IEEE 754 ▪ <i>exemples</i> : 19, 19.05, 2e-8 ▪ valeurs spéciales : NaN (Not-a-Number), Infinity et -Infinity
String	▪ séquence de valeurs entières non-signées 16-bits ▪ représente des données textuelles (chaines de caractères) ▪ <i>exemples</i> : "mon texte" ou 'mon texte'
Object	▪ collection de propriétés

- **les types sont représentés par des objets intégrés**
- tout objet dérivé de ces objets n'est pas considéré comme un nouveau type

Types intégrés et opérateur unaire `typeof`

Type	Résultat de l'opérateur <code>typeof</code>
Undefined	"undefined"
Null	"object"
Boolean	"boolean"
Number	"number"
String	"string"
Object	"object" "function" type hôte

- permet de tester le type des objets, plusieurs syntaxes

```
var i = 10;  
var s = typeof i;  
console.log(s);           // affiche « number »  
i = true;  
console.log(typeof i);    // affiche « boolean »  
i = null;  
console.log(typeof(i));   // affiche « object »
```

Conversion automatique de types

Conversion vers
le type **Boolean**

Type d'origine	Résultat
Undefined	false
Null	false
Number	false si le nombre vaut +0, -0 ou NaN; true sinon
String	false si la chaîne est vide (de longueur nulle); true sinon

Conversion vers
le type **Number**

Type d'origine	Résultat
Undefined	NaN
Null	+0
Boolean	1 si le booléen vaut true; +0 si il vaut false
String	converti la chaîne en suivant la syntaxe des nombres

Conversion vers
le type **String**

Type d'origine	Résultat
Undefined	"undefined"
Null	"null"
Number	nombre tel qu'il aurait pu être écrit dans le code
Boolean	"true" "false"

Objets

Objet	collection non-ordonnée de paires (propriété,valeur)
Propriété	objet, comprend les méthodes (fonction)
Valeur primitive	types intégrés : Undefined, Null, Boolean, Number, String
Types intégrés	Undefined, Null, Boolean, Number, String, Object
Objets intégrés	Object, Function, Array, String, Boolean, Number, Math, Date, RegExp, JSON, Error, EvalError, RangeError, ReferenceError, SyntaxError, TypeError, URIError

- une propriété est représentée par une variable associée à l'objet
- tout objet dérive de l'objet **Object**
- les fonctions sont des objets

Créer un nouvel objet

- forme littérale

```
var point = { x:10, y:15, z:20 };
```

ensemble de paires **nomPropriété:valeur**
séparées par des virgules

Accès aux propriétés

- par l'**opérateur .**
- ou par l'**opérateur crochet : []**

```
console.log(point.x); // affiche la valeur de x dans la console  
point.y = 15.3;       // modifie la valeur de y  
point['z'] = 12;       // modifie la valeur de z
```

attention aux cotes (simples ou doubles)

Ajout de nouvelles propriétés

- à la volée

```
var point = { x:10, y:15, z:20 };  
  
point.w = 1; // ajout d'une nouvelle paire  
// ou bien  
point['w'] = 1;
```

→ un objet est un tableau associatif

Tableau associatif / dictionnaire / table d'association

- structure de donnée
- associe un ensemble de valeurs à un ensemble de clés
- les clés sont indexées

clé	x	y	z	w
valeur	10	15	20	1

Nom des propriétés

- caractère
- chaîne de caractères
- nombre
- tout objet

```
var monObjet = {};          // objet sans propriété spécifique
var nb = 2, ch = "maclef";

monObjet.nb1 = 1;           // opérateur point
monObjet["ECMAScript!"] = true; // opérateur crochet
monObjet["prop spc"] = "chaîne"; // avec espace(s)
monObjet[5] = 12;           // un nombre
monObjet[nb] = 15.5;        // à partir d'1 variable
monObjet[ch] = "valeur";    // à partir d'1 variable
monObjet[""] = nb;          // chaîne vide
```

- tout nom avec un espace, un tiret, commençant par un chiffre, ou un mot réservé ES/JS, n'est utilisable qu'avec l'opérateur crochet

par exemple `monObjet.2prop` n'est pas valide
mais `monObjet["2prop"]` est valide

Créer un nouvel objet - constructeur & opérateur new

- **rappel** : tout objet est un objet **Object**
- tout objet possède un **constructeur**
 - fonction permettant d'initialiser un objet, i.e. ses propriétés
 - appelée lors de la création de l'objet
 - porte le nom de l'objet
- avec le constructeur de l'objet **Object** et le mot-clé **new**

```
// objet sans propriété spécifique  
var point = new Object();  
  
// ajout de propriétés spécifiques  
point.x = 10;  
point.y = 20;  
point.z = 15;
```

Objets

- **rappel** : tout objet est un objet **Object**
- tout objet possède les propriétés de **Object** (en fait pas toutes)
- et ses propres propriétés
- spécialisation

Référence de l'objet **Object**

- MDN
developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Objets_globaux/Object
- observez les propriétés y compris les méthodes
- plusieurs types de propriétés (nous reviendrons sur ce point)

Fonctions - déclaration

- avec le mot-clé **function**
- avec ou sans argument

Sans argument

```
function ma_fonction()
{
  ...
}
```

nom de la fonction

Avec argument(s)

```
function ma_fonction(arg1, arg2, ...)
{
  ...
}
```

arguments

```
function affichePoint(pt) // déclaration d'1 fonction
{
  console.log("point (" + point.x + "," +
              point.y + "," + point.z + ")");
}

var point = {x: 5, y: 10, z: -30 }; // création d'1 point

affichePoint(point); // appel de la fonction
```

Que se passe-t-il ?

```
1  function affichePoint(pt)  // déclaration d'1 fonction
2  {
3      console.log("point (" + pt.x + "," + ← opérateur de
4                      pt.y + "," + pt.z + ")");  concaténation
5  }                                             de chaînes
6
7  var point = {x: 5, y: 10, z: -30 };  // création d'1 point
8
9  affichePoint(point);  // appel de la fonction
```

Lors de l'exécution

- l'objet **point** est créé (l.7)
- la fonction **affichePoint** est exécutée avec l'objet **point** comme argument (l.9)
 - la variable **pt** représente maintenant l'objet **point** (l.1)
 - la méthode **log** de l'objet **console** est exécutée avec une chaîne de caractères comme argument (l.3 et 4)
 - affichage de « **point (5,10,-30)** » dans la console web

Fonction - valeur de retour

- avec le mot clé **return**
 - suivi d'une expression : retourne la valeur de l'expression
 - seul : retourne **undefined**
- sans le mot clé **return**
 - retourne implicitement **undefined**

```
function translate(pt,vec)  // fonction translation
{
    var pt_res = { x: pt.x+vec.x,
                   y: pt.y+vec.y,
                   z: pt.z+vec.z };
    return pt_res;
}

var point = {x: 5, y: 10, z: -30 };    // 1 point
var vecteur = {x: 5, y: 0, z: 2 };    // 1 vecteur

var pt_tr = translate(point,vecteur);  // appel fonction
```


Que se passe-t-il ?

```
1  function translate(pt,vec)  // fonction translation
2  {
3      var pt_res = { x: pt.x+vec.x,
4                      y: pt.y+vec.y,
5                      z: pt.z+vec.z };
6      return pt_res;
7  }
8  var point = {x: 5, y: 10, z: -30 };    // 1 point
9  var vecteur = {x: 5, y: 0, z: 2 };    // 1 vecteur
10 var pt_tr = translate(point,vecteur);  // appel fonction
```

Lors de l'exécution

- les objets **point** et **vecteur** sont créés (l.8 et 9)
- la fonction **translate** est exécutée avec ces objets comme arguments (l.10)
 - les variables **pt** et **vec** représentent **point** et **vecteur** (l.1)
 - un objet **pt_res** est créé à partir des arguments (l.3,4,5)
 - cet objet est retourné par la fonction (l.6)
- un objet **pt_tr** est créé et affecté à l'objet retourné par la fonction (l.10) i.e. { **x: 10, y: 10, z: -28** }

Variables - résolution de portée

▪ variable globale

- valide dans le script entier
- valide dans les autres scripts après la déclaration de la variable
- déclaration en dehors de toute fonction avec ou sans le **var**
- déclaration dans toute fonction sans le mot-clé **var**

▪ variable locale

- valide dans une fonction
- déclaration dans une fonction avec le mot-clé **var**
- arguments d'une fonction

```
function translate(pt,vec) // pt et vec variables locales
{
    // à la fonction
    var pt_res = { x: pt.x+vec.x, // pt_res locale
                  y: pt.y+vec.y, // à la fonction
                  z: pt.z+vec.z };
    return pt_res;
}
var point = {x: 5, y: 10, z: -30 }; // globale
var vecteur = {x: 5, y: 0, z: 2 }; // globale
var pt_tr = translate(point,vecteur); // globale
```

Variables - résolution de portée

▪ attention aux variables globales

- valide dans le script entier
- valide dans les autres scripts après la déclaration de la variable
- déclaration en dehors de toute fonction avec ou sans le **var**
- déclaration dans toute fonction sans le mot-clé **var**

```
function translate(pt,vec)  // pt et vec locales
{
    point = { x: pt.x+vec.x,      // point est globale
              y: pt.y+vec.y,      // car pas de var
              z: pt.z+vec.z };    // écrase l'ancien objet
    return point;
}

var point = {x: 5, y: 10, z: -30 };    // globale
var vecteur = {x: 5, y: 0, z: 2 };    // globale
var pt_tr = translate(point,vecteur);  // globale
```

→ éviter d'utiliser des variables non déclarées avec var

Variables - résolution de portée

▪ variables locales priment sur les globales

- variable locale obligatoirement déclarée avec le mot-clé **var**
- nom d'une variable locale identique à celui d'une variable globale
- la variable est indépendante de la globale

```
function translate(pt,vec)  // pt et vec locales
{
    var point = { x: pt.x+vec.x,          // point est locale
                  y: pt.y+vec.y,          // différente de
                  z: pt.z+vec.z };        // point global
    return point;                // malgré leur nom
}

var point = {x: 5, y: 10, z: -30 };      // globale
var vecteur = {x: 5, y: 0, z: 2 };      // globale
var pt_tr = translate(point,vecteur);    // globale
```

→ éviter d'utiliser des variables non déclarées avec **var**

4 - Objets

Objets - méthodes

- **propriétés qui sont des fonctions**
 - respectent la syntaxe des propriétés
 - et celle des fonctions
 - avec une légère différence

```
var point = {x: 5, y: 10, z: -30 };    // objet point

point.translate = function(vec)        // ajout méthode
{                                     // translate
    this.x = this.x + vec.x;          // directement le
    this.y = this.y + vec.y;          // point
    this.z = this.z + vec.z;
    return this;                      // et le retourne
}

point.translate({ x:2, y:0, z:-1 });   // appel
```

- le mot-clé **this** désigne l'objet courant, ici **point**

Objets - réutilisabilité

- l'objet point que nous avons construit n'est **pas réutilisable**
 - ne permet pas de créer un nouvel objet point sans réécrire l'ensemble des instructions
- utilisation d'un **constructeur** explicite
 - sinon celui de l'objet **object** est appelé

Objets - constructeur

- fonction
- permet de définir comment un objet est créé / initialisé
- porte le nom de l'objet

```
function Point(x, y, z)                // constructeur de Point
{
    this.x = x;                        // création des propriétés
    this.y = y;
    this.z = z;
}
```

Objets - constructeur

- appel pour créer un objet dérivé = instanciation d'un objet
 - avec le mot-clé **new**

```
function Point(x, y, z)           // constructeur de Point
{
    this.x = x;                   // création des propriétés
    this.y = y;
    this.z = z;
    this.translate = function(vec) // méthode translate
    {
        this.x = this.x + vec.x;
        this.y = this.y + vec.y;
        this.z = this.z + vec.z;
        return this;
    }
}

var pt1 = new Point(10,2,-1);     // création d'un point
var pt2 = new Point(3,5,8);       // création d'un point
pt1.translate(pt2);               // translation, modif pt1
```


Objets

- attention il existe plusieurs syntaxes pour créer des objets
- elles sont **complémentaires**

```
function Point(x, y, z)           // constructeur de Point
{
    this.x = x;                   // création des propriétés
    this.y = y;
    this.z = z;
    this.translate = function(vec) // méthode translate
    { ... }
}

var pt1 = new Point(10,2,-1);     // création d'un point
var pt2 = new Point(3,5,8);       // création d'un point

pt1.w = 2.5;                      // ajout d'une propriété spécifique
                                   // à l'objet pt1 instance de Point

console.log(pt2.w);               // affiche undefined
```

Objets - Réutilisabilité

- la construction de l'objet Point n'est pas optimale
- les méthodes seront créées autant de fois qu'il y a de Point instanciés

```
function Point(x, y, z)           // constructeur de Point
{
    this.x = x;                   // création des propriétés
    this.y = y;
    this.z = z;
    this.translate = function(vec) // méthode translate
    {
        this.x = this.x + vec.x;
        this.y = this.y + vec.y;
        this.z = this.z + vec.z;
        return this;
    }
}

var pt1 = new Point(10,2,-1);     // création d'un point
var pt2 = new Point(3,5,8);       // création d'un point
pt1.translate(pt2);               // translation, modif pt1
```

Objets - prototype

- chaque objet possède une propriété **prototype**
- elle permet de déclarer des propriétés (dont méthodes) identiques pour toutes les instances **sans les dupliquer**

```
function Point(x, y, z)           // constructeur de Point
{
    this.x = x;                   // création des propriétés
    this.y = y;
    this.z = z;
}

Point.prototype.translate = function(vec) // méthode
{
    this.x += vec.x;
    this.y += vec.y;
    this.z += vec.z;
    return this;
}

// utilisation identique
```

Prototype et héritage

- chaque objet possède une propriété **prototype**
- **prototype** permet de transmettre les propriétés (dont méthodes) des type d'objets par **héritage**
- par exemple tout nouveau type d'objet comme **Point** hérite d'**Object**
 - **Point** possède aussi les propriétés de **Object.prototype**
 - par exemple la méthode **Object.prototype.toString()** qui transforme l'objet en une chaîne de caractères

```
var pt1 = new Point(10,2,-1); // création d'un point  
  
console.log(pt1.toString());
```

Résultat : [object Object]

Prototype et surcharge

- chaque objet possède une propriété **prototype**
- **prototype** permet de transmettre les propriétés (dont méthodes) des type d'objets par **héritage**
- les propriétés (dont méthodes) héritées peuvent être **surchargées** pour définir un comportement spécifique à l'objet

```
Point.prototype.toString = function() // surcharge de la
{                                     // méthode d'Object
    return '(' + this.x + ',' + this.y + ',' + this.z + ')';
}

var pt1 = new Point(10,2,-1); // création d'un point

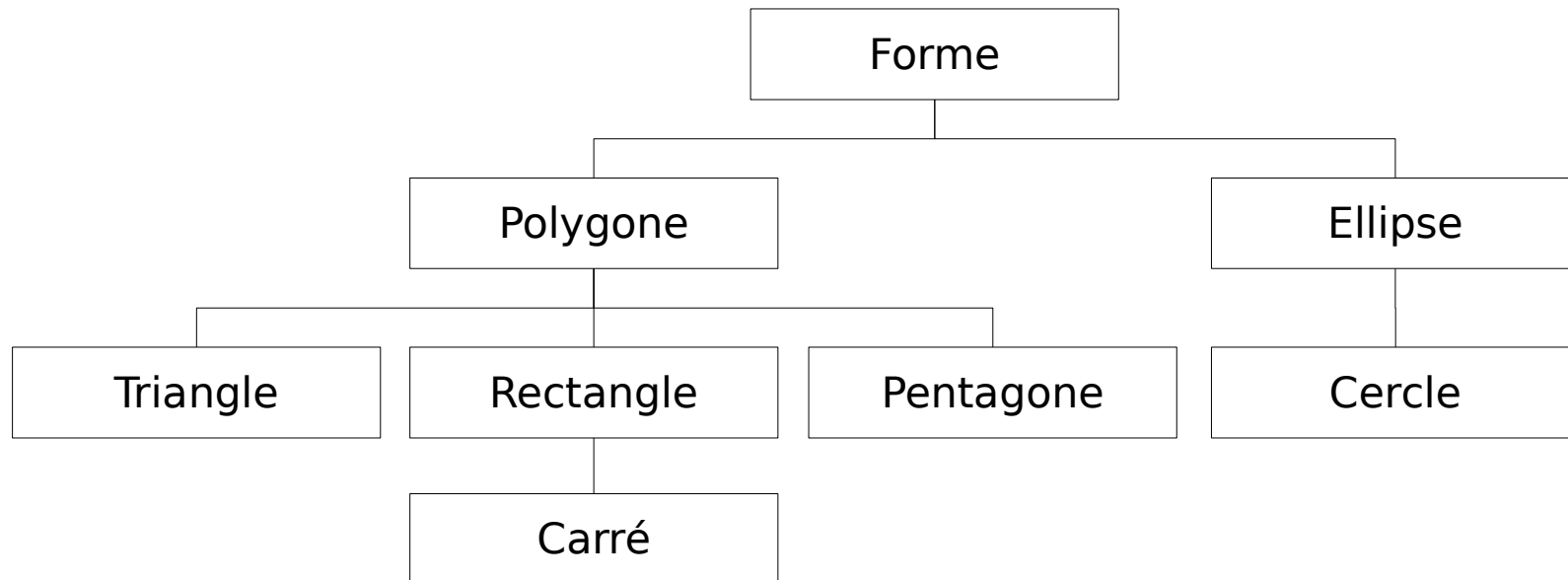
console.log(pt1.toString());
```

Résultat : (10,2,-1)

→ spécialisation de la méthode **toString** pour **Point**

Héritage

- l'**héritage** en ECMAScript est **émulé**
- plusieurs façons de coder l'héritage
- plusieurs librairies existent (*zInherit*, *xbObjects*, ...)
 - notion de classes
 - héritage multiple, accès aux classes parentes, ...



Héritage - Méthode usuelle

- héritage des propriétés de l'objet : méthode **call**
- héritage des propriétés de **prototype** : instantiation

```
function Polygon(sides)
{
    this.sides = sides
}
Polygon.prototype.getArea = function() { }
```

objet avec méthodes
définies via prototype

```
function Triangle(base, height)
{
    Polygon.call(this, 3)
    this.base = base
    this.height = height
}
```

héritage des propriétés

```
Triangle.prototype = new Polygon;
Triangle.prototype.getArea = function()
{
    return 0.5 * this.base * this.height
}
```

héritage du prototype

surcharge

```
var poly = new Polygon(2);
console.log(poly.getArea()); // undefined
var trgl = new Triangle(2, 3);
console.log(trgl.getArea()); // 3
```

Héritage - Méthode usuelle

- attention l'héritage du prototype appelle le constructeur sans argument
 - crée une propriété **prototype.sides** valant **undefined**
- tester dans le constructeur si aucun argument n'est passé

```
function Polygon(sides)
{
    if (arguments.length == 1) // test des arguments
    {
        this.sides = sides
    }
}
Polygon.prototype.getArea = function() { }

function Triangle(base, height)
{
    Polygon.call(this, 3)
    this.base = base
    this.height = height
}
Triangle.prototype = new Polygon;
Triangle.prototype.getArea = function()
{
    return 0.5 * this.base * this.height
}
```

→ héritage du prototype

Approche recommandée pour créer un objet

- dans son **constructeur** : déclarer ses **propriétés**
- dans le **script** : déclarer ses **méthodes** via sa propriété **prototype**
 - avantage : ne crée qu'une seule fois les méthodes

Déréférencement

- lorsqu'il n'existe **plus de référence** à un objet, il est dit **déréférencé**
- forcer le déréférencement d'un objet en affectant **null** à toutes les références de l'objet

```
var etudiant1 = new personne("Dupont", "Paul", 19, "Luc", "Marie");  
etudiant1 = null;
```

Suppression de propriétés hôtes

- opérateur **delete**

```
var etudiant1 = new personne("Dupont", "Paul", 19, "Luc", "Marie");  
delete(etudiant1.nom);
```

5 - Arguments des fonctions

Fonction - Objet Function

- chaque fonction est un objet natif **Function**
 - propriétés et méthodes

```
var nomFonction = new Function(arg1,arg2,...,corps_fonction);
```

arguments
optionnels

corps de la fonction

```
// déclaration d'une fonction  
var square = new Function('nombre','return nombre*nombre');  
console.log(square(7));
```

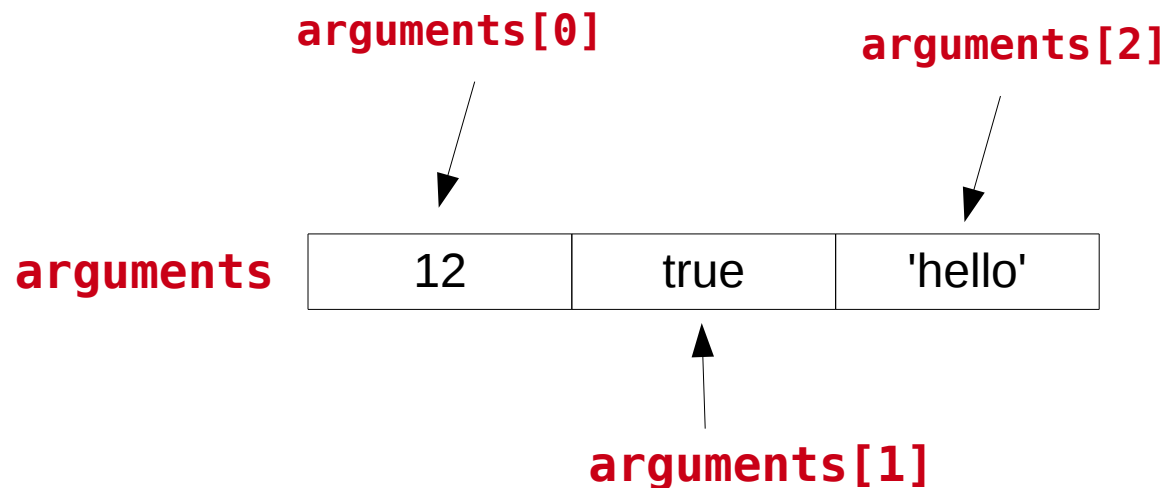
Fonctions - arguments (ou paramètres)

- les arguments sont représentés par une **variable locale** à la fonction nommée **arguments**
- **arguments** est un objet similaire à **Array** (mais en lecture seule)

```

function myFunction(arg1, arg2, arg3)
{
    // ...
}

myFunction(12, true, 'hello');
  
```



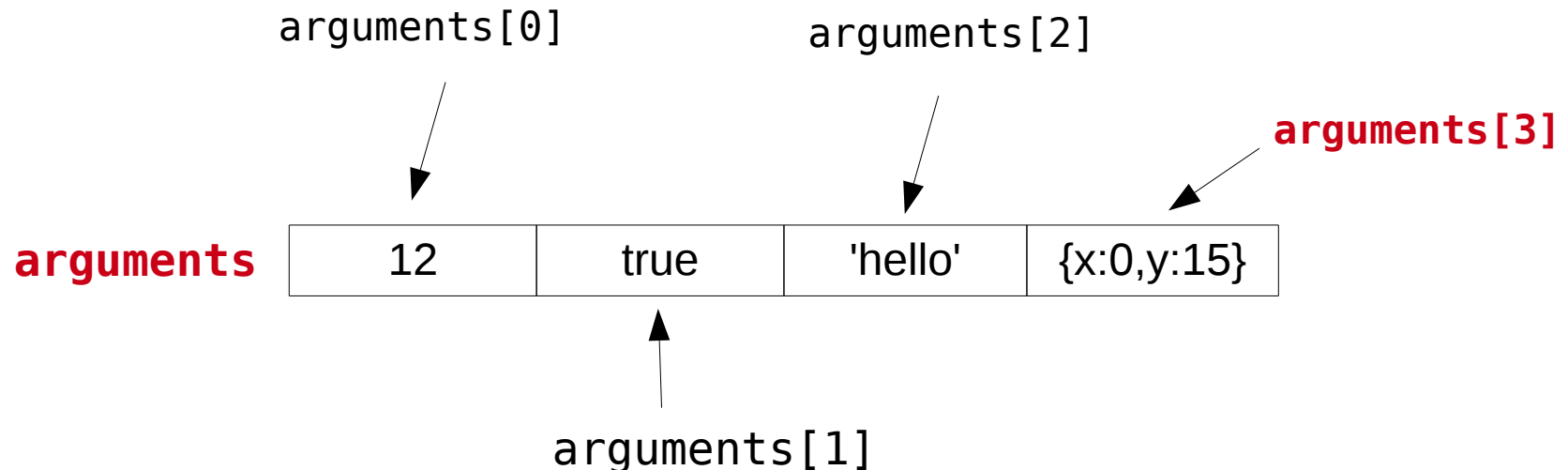
Fonctions - arguments (ou paramètres)

- possible de **passer plus ou moins d'arguments** relativement à la déclaration
- **arguments** permet de récupérer les arguments en plus

```

function myFunction(arg1, arg2, arg3)      // 3 déclarés
{
    // ...
}

myFunction(12, true, 'hello', {x:0, y:15}); // 4 passés
  
```



Fonctions - arguments (ou paramètres)

- utilisation de l'objet **arguments**
- parcourir les éléments d'un tableau : boucle **for**
- nombre d'arguments : **arguments.length**

1ère version : boucle **for** classique

```
function myFunction()  
{  
    for (var i = 0; i < arguments.length; i++)  
    {  
        // faire qq chose avec arguments[i]  
        console.log(arguments[i])  
    }  
}  
  
myFunction(12,true,'hello',{x:0,y:15});    // 4 passés
```

Résultat : 12
true
hello
({x:0, y:15})

Fonctions - arguments (ou paramètres)

- parcourir les éléments d'un tableau : boucle **for**

1ère version : boucle **for** classique

nombre d'arguments passés < au nombre d'arguments attendus

```
function myFunction(arg1, arg2, arg3) // 3 déclarés
{
    for (var i = 0; i < arguments.length; i++)
    {
        // faire qq chose avec arguments[i]
        console.log(arguments[i])
    }
}

myFunction(12,true); // 2 passés
```

Résultat : 12
true

Fonctions - arguments (ou paramètres)

- parcourir les éléments d'un tableau associatif : **for**

2ème version : boucle **for** avec **in**

```
function myFunction(arg1,arg2,arg3) // trois déclarés
{
    for (var i in arguments)
    {
        // faire qq chose avec arguments[i]
        console.log(arguments[i])
    }
}

myFunction(12,true); // deux passés
```

Résultat : 12
true

→ les deux types de boucles ont ici le même comportement

Fonction - Définition et arguments

- **préciser les arguments attendus**
 - plus explicite
- **vérifier qu'ils sont fournis**
 - gestion des erreurs et / ou spécialisation des fonctions relativement au nombre d'arguments passés
 - `if (argi == undefined) // argument manquant`
- **fonctions sans arguments déclarés**
 - lorsque la fonction doit traiter un nombre variable d'arguments